

SYSTEM INTEGRATION PRACTICES

System Integration Life Cycles



Instructor: Huy Nguyen Dang Quang, Msc

Email: huyndq@duytan.edu.vn

REF: Shawn A. Butler, Ph.D

Senior Lecturer, Executive Education Program

Institute for Software Research

Carnegie Mellon University

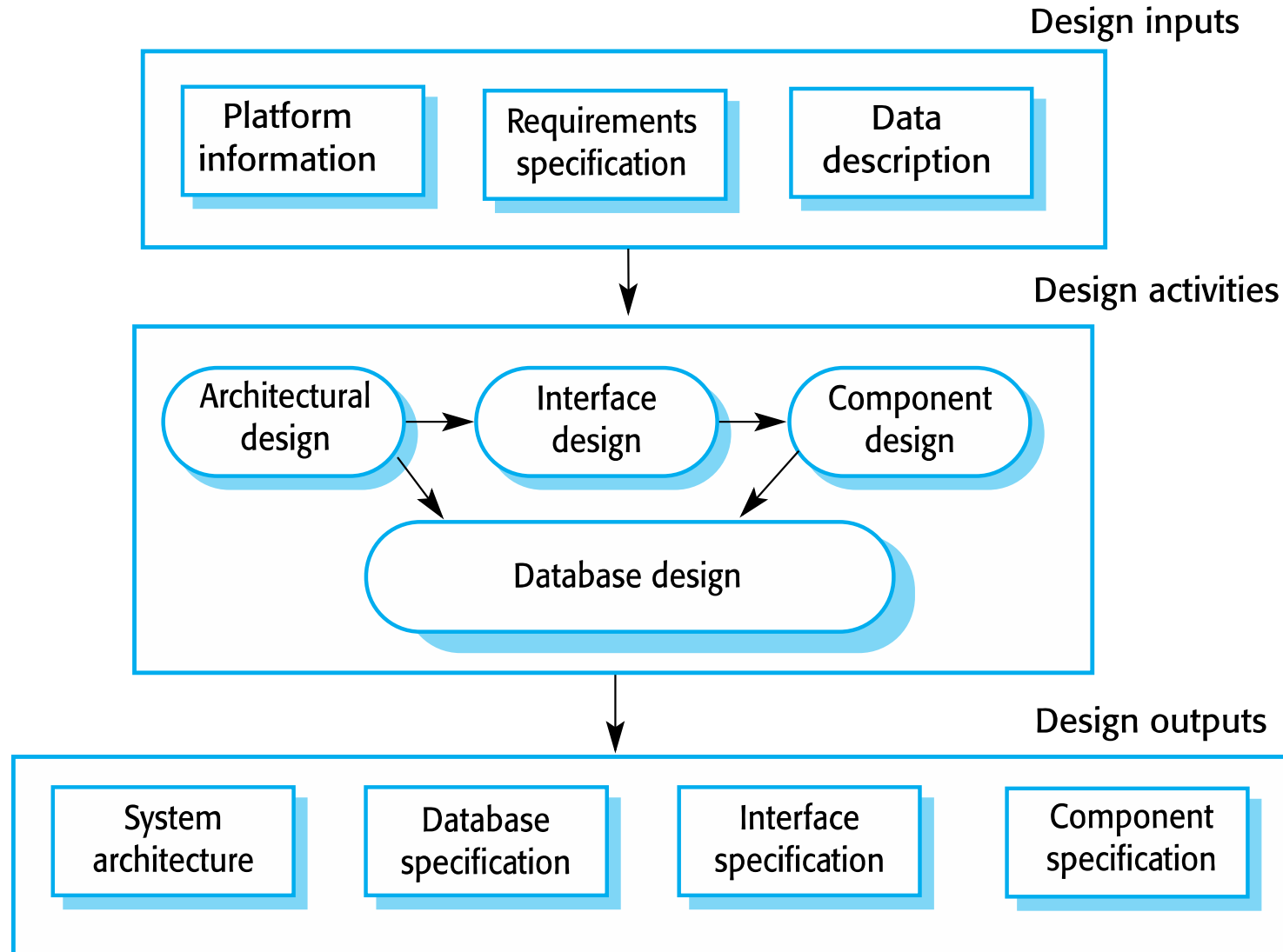
Lecture Objectives

- Describe the system integration life cycles and potential “**red flags**” in each phase
- Understand how system integration life cycles differ from traditional software development life cycles
- Learn how to select a system development life cycle for a system integration project

The software process

- A structured set of activities required to develop a software system.
- Many different software processes but all involve:
 - **Specification** – defining what the system should do;
 - **Design and Implementation** – defining the organization of the system and implementing the system;
 - **Validation** – checking that it does what the customer wants;
 - **Evolution** – changing the system in response to changing customer needs.
- A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective.

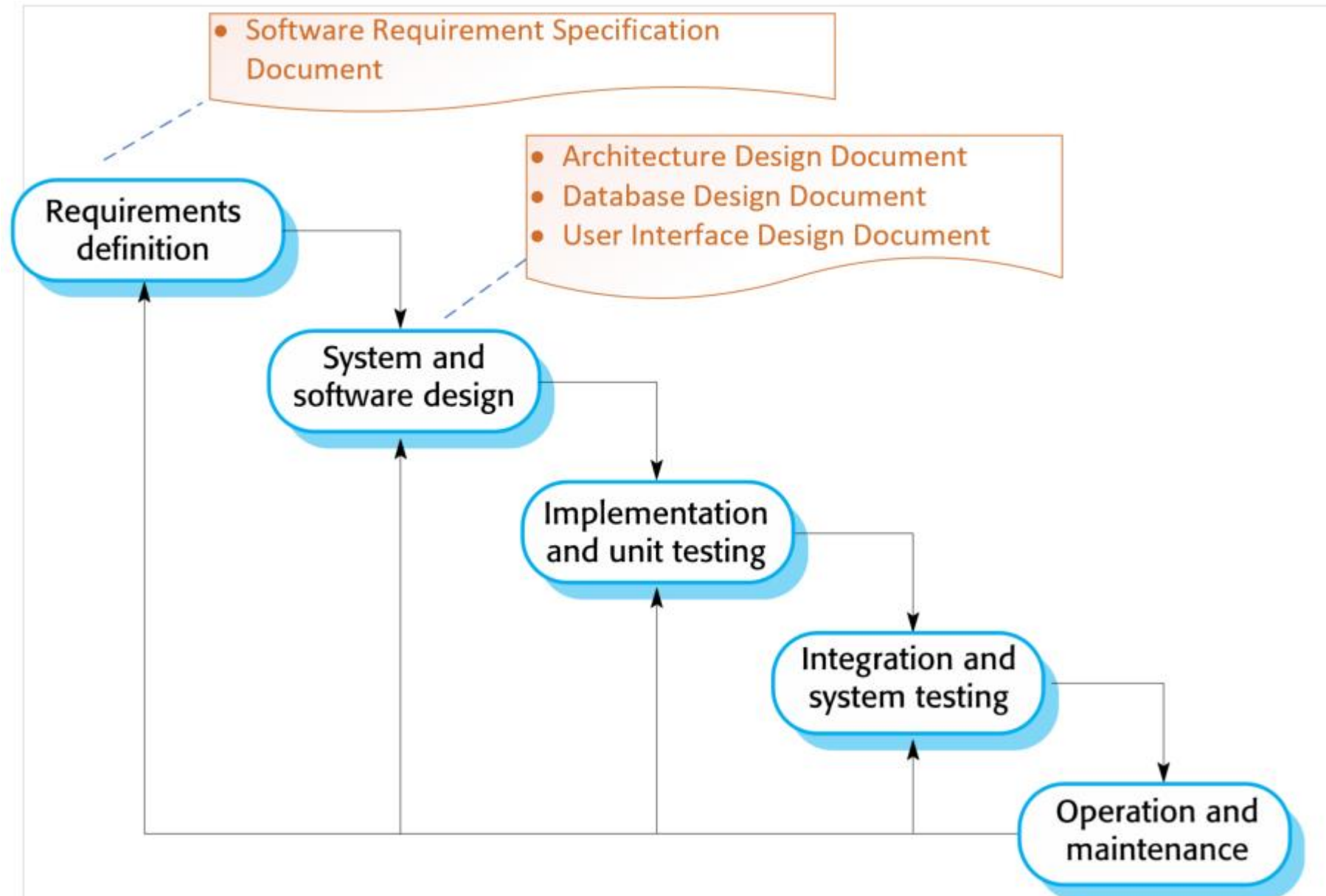
Review: A general model of the design process



- **System Integration Life Cycle (SILC)** refers to the step-by-step management process for integrating different systems and software into a unified, functional system. SILC is essential for projects involving multiple components, including **legacy systems** and **modern services**. Unlike SDLC (Software Development Life Cycle), SILC focuses not only on software development but also on **connecting, integrating, testing, and maintaining** various systems.

- **Legacy System Management:** One of the biggest challenges is integrating old systems with new technologies.
- **Dependency Management:** SILC ensures that components work together smoothly.
- **Security and Risk Management:** Assess and manage risks throughout the life cycle.

SILC: The waterfall model



Advantages of the Waterfall Process

Easy to Understand and Manage:

- The Waterfall process has a clear structure, with each phase well-defined, making it easy for team members to grasp and implement.
- It is straightforward to manage progress, allocate tasks, and monitor outcomes at each stage.

Suitable for Projects with Clear Requirements:

- It works well when project requirements are clear, stable, and unlikely to change during development.
- Helps avoid confusion and errors during transitions between phases.

Good Documentation:

- Comprehensive documentation at every phase facilitates maintenance, debugging, or handing over the project to another team.

Time-Efficient for Small Projects:

- For small-scale projects, it can be executed quickly since repetitive iterations are not required.

Disadvantages of the Waterfall Process

Lack of Flexibility:

- It is challenging to accommodate changes once the project has started, as the phases are tightly interdependent.
- Modifications in later stages often require significant time and cost.

Late Detection of Errors:

- Issues or design flaws are typically identified only during the testing phase, making corrections complex and costly.

Unsuitable for Complex or Unclear Projects:

- It is not ideal for projects with unstable requirements or those requiring frequent changes.

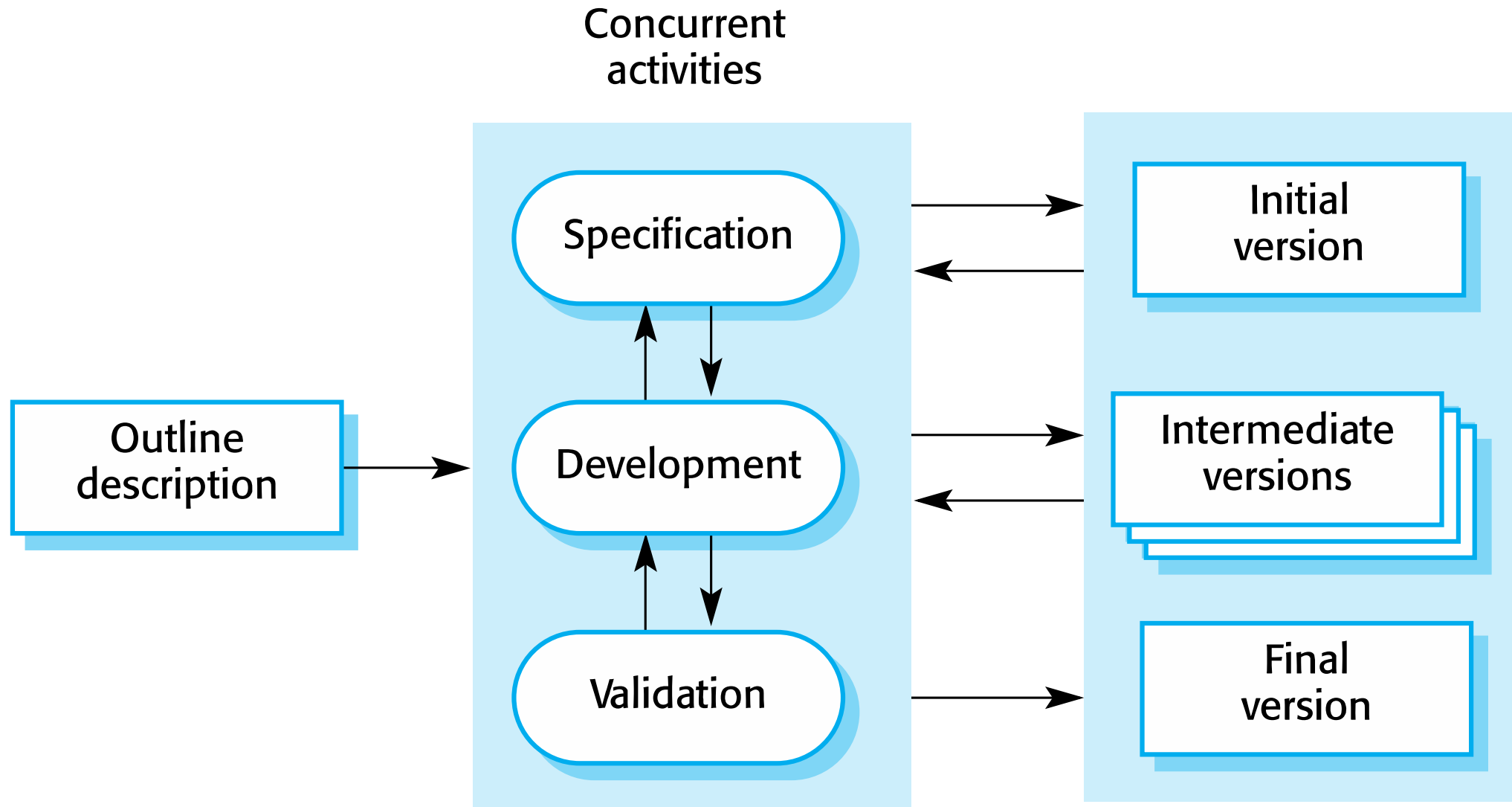
Long Time-to-Market:

- The final product is delivered only after all phases are completed, making it impossible to release partial components early for feedback.

Difficult to Meet Customer Needs:

- Since customers only see the final product, any required changes or dissatisfaction at the end stage are challenging to address.

SILC: Incremental development



Advantages of Incremental Development

Early and Frequent Releases:

- Small portions of the product are released early, enabling quicker time-to-market and user feedback collection.

Adaptability to Changes:

- Each iteration focuses on a small portion, making it easier to adjust to changing requirements or customer feedback.

Reduced Risk:

- Since the product is developed in increments, the risk of complete project failure or major issues is significantly minimized.

Easier Management:

- Dividing the project into smaller parts makes it more manageable in terms of progress and resource allocation.

Testing of Individual Parts:

- Each increment is tested independently, allowing for early issue detection and reducing repair costs.

Enhanced Customer Satisfaction:

- Delivering continuous and functional increments builds customer trust and satisfaction.

SILC: Disadvantages of Incremental Development

Complexity in Scalability:

- As the product grows larger, managing and integrating the increments can become increasingly difficult.

The process is not visible:

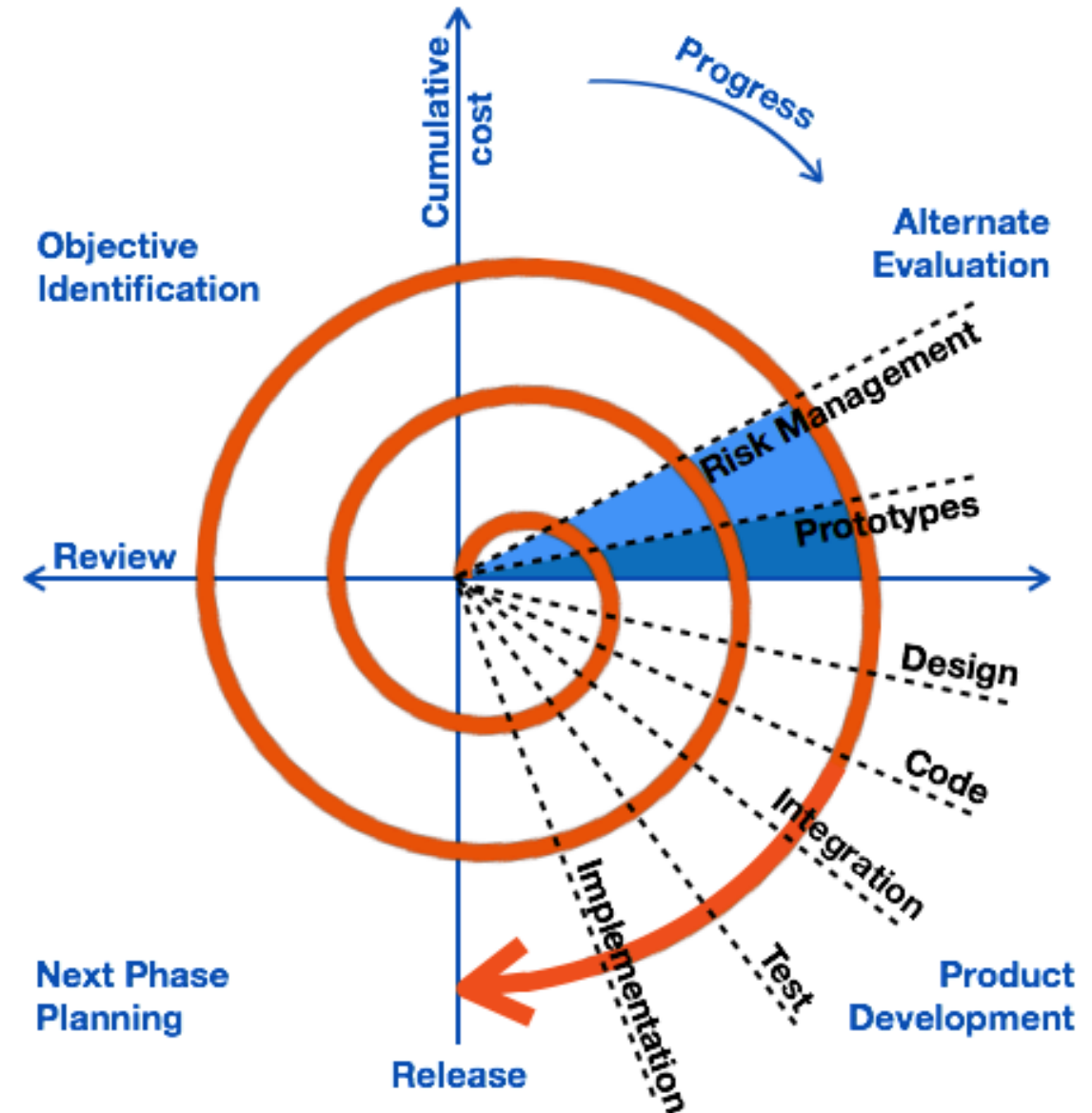
- Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.

System structure tends to degrade as new increments are added:

- Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.
- Requires ongoing **refactoring** to maintain code structure and quality

SILC: Spiral

- **The Spiral Model** is a software development model that combines iterative and phased approaches. This model focuses on risk management and project development through multiple iterations (or spirals).



SILC: Phases in the Spiral Model:

1.Objective Setting:

- Define goals for the iteration, analyze requirements, and identify potential risks.

2.Risk Assessment:

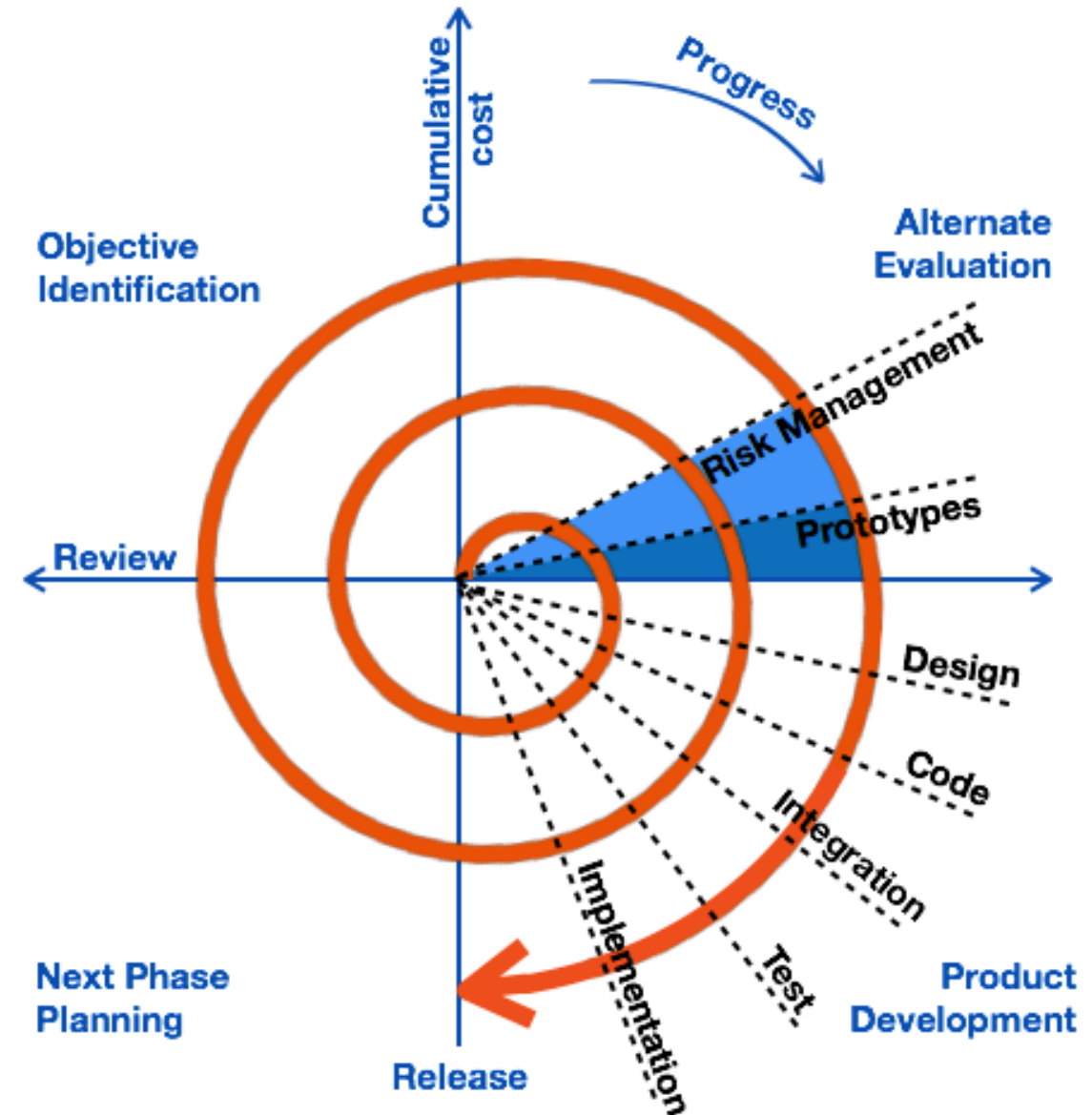
- Evaluate and prioritize risks, then propose solutions to mitigate or eliminate them.

3.Development and Testing:

- Design, develop, and test parts of the product in each iteration.

4.Review and Planning:

- Customers review the results of the iteration, provide feedback, and plan for the next iteration.



Advantages:

- Effective risk management.
- Flexible to requirement changes.
- Enables early delivery of partial product components.
- Suitable for large, complex projects with unclear requirements.

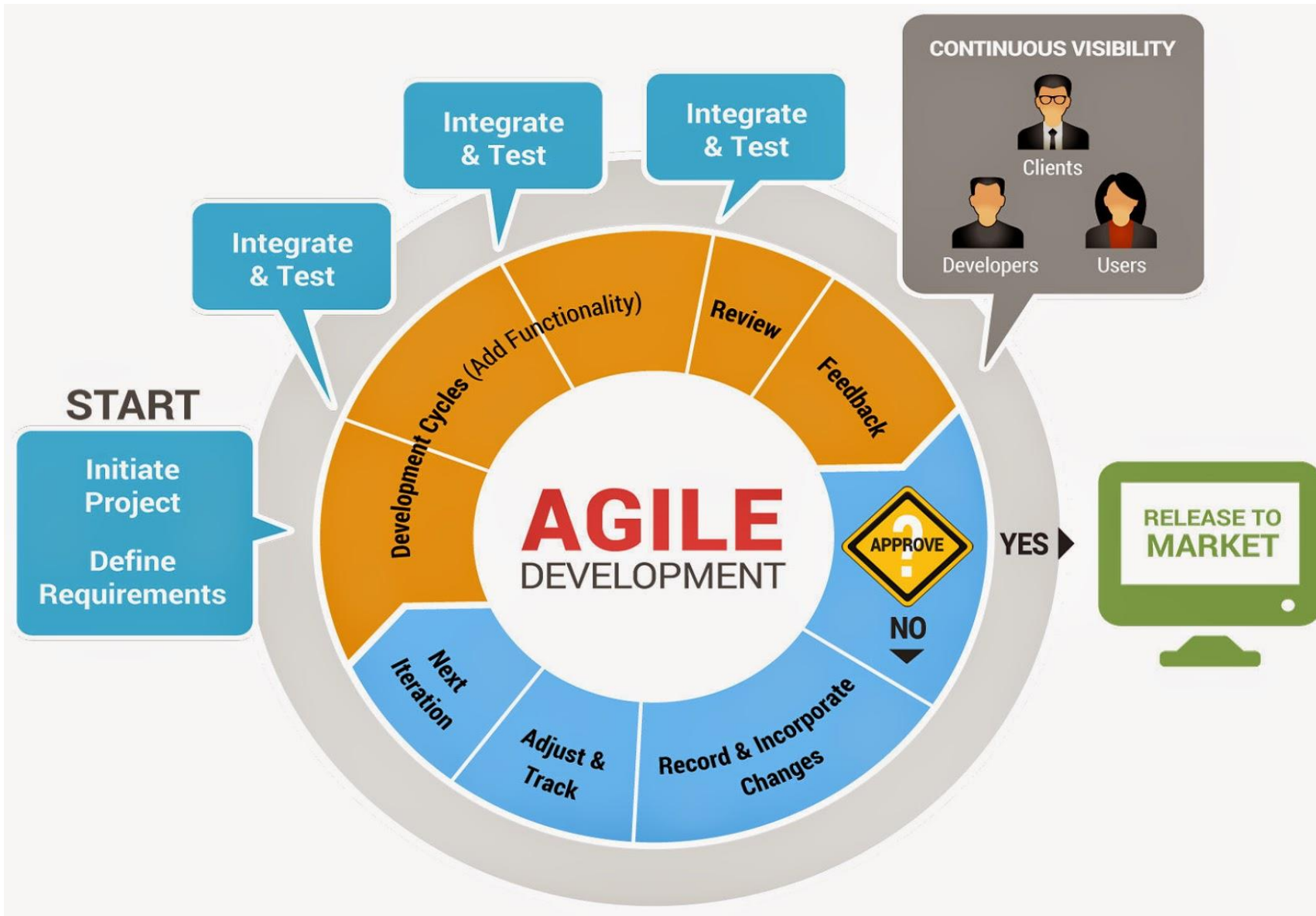
Disadvantages:

- Requires experienced risk management and analysis.
- Can be time-consuming and costly if not well-controlled.
- More complex than traditional models like Waterfall.

When to Use:

- The Spiral Model is best suited for large-scale, long-term projects with many potential risks or frequently changing requirements.

SILC: Agile



SILC: Agile

- Emphasis on user involvement
- Very short increments develop
- Requirements and vision may not be well developed
- Highly flexible
- Design into a corner
- Difficult to integrate security
- Often no documentation

SILC: Agile

Advantages of Agile:

- Flexible with changing requirements.
- Improves communication and collaboration between team members and customers.
- Quick feedback allows adjustments to make the product better aligned with user needs.
- Products are developed quickly and usable after each iteration.

Disadvantages of Agile:

- Can be challenging to manage large-scale projects without tight control.
- Sometimes lacks detailed documentation.
- Requires continuous customer involvement, which can be challenging in certain cases.

SILC: Agile

Is Agile a Process or methodology?

????

SILC: Scrum

The Agile: Scrum Framework

Inputs from Executives,
Team, Stakeholders,
Customers, Users



Product Owner



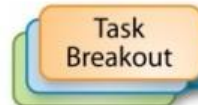
The Team



Product Backlog

Team selects
starting at top
as much as it
can commit
to deliver by
end of Sprint

Sprint
Planning
Meeting



Sprint
Backlog



1-4 Week
Sprint

Sprint end date and
team deliverable
do not change



Scrum
Master



Burndown/up
Charts

Every
24 Hours



Daily Scrum
Meeting



Sprint Review



Finished Work



Sprint
Retrospective

SILC

**All SDLC's go through every
phase the order, emphasis, and
implementation may differ**

System and Software Integration Life Cycles

Primary Objectives:

- **Software Development Process:**

- Focuses on creating software from scratch, including activities such as requirements analysis, design, coding, testing, and deployment.
- The main goal is to build a software product that meets specific customer or user requirements.

- **System Integration Process:**

- Focuses on combining multiple components, including software, hardware, databases, and other systems, to create a complete, functional system.
- The objective is to ensure that all components work seamlessly together to meet the overall system requirements.

System and Software Integration Life Cycles

Scope:

- **Software Development Process:**
 - Narrower in scope, dealing only with the creation of software.
 - Primarily addresses technical and logical aspects of software.
- **System Integration Process:**
 - Large scope, involving both software and non-software components (*such as physical devices, networks, or third-party services*).
 - Requires collaboration across various domains (*hardware, software, databases, networks, etc.*).

System and Software Integration Life Cycles

Requirements and Specification:

- Must determine which legacy systems to access
- SILC must focus on how to access and integrate data from legacy systems.
- Determine how to access the legacy system
- Identify or coordinate changes to legacy systems
- Resolve redundancy issues, i.e., which system is the data source
- Identify impact of updates if applicable
- Identify process changes
- Identify maintenance and change processes
- Resolving Data Redundancy

System and Software Integration Life Cycles

- Testing

- Far more complicated than developing a single application
- Design tests that are within system integrator's control
- Data does not adhere to specifications
- Impossible to test all scenarios – must select carefully
- Can't always replicate system in a test environment
- Difficult to problem solve errors because of complexity
- **More Complex Testing:** Testing needs to cover both interfaces and data flows between multiple systems (Example: Not all scenarios can be tested; prioritize key cases instead)
- **Automated Testing Tools:** Use **Selenium** or **Postman** to automate API testing.

System and Software Integration Life Cycles

Maintenance and Retirement

- Maintenance complex and requires careful coordination with legacy owners
- Testing bug fixes and adding functionality makes testing more challenging
- Changes in legacy systems often affect overall system and aren't usually thoroughly tested
- Legacy systems may need to remain in place until new systems are completely integrated

Selecting a SDLC Model for Project Integration:

Choosing the right SDLC model is crucial, depending on specific project factors such as **requirements, resources, team, and goals**. Key considerations include:

- **Ability to handle rapidly changing requirements:** Agile is ideal for projects with frequent changes due to its flexibility.
- **Cost-effective management of changes:** Agile and Iterative models lower refactoring costs compared to Waterfall.
- **Focus on quality:** Spiral and V-Model are suitable for projects requiring rigorous(*chặt chẽ*) quality testing.
- **Complexity and scale management:** Spiral or Waterfall suits large, complex projects, while Agile works for smaller ones.
- **Customizability and flexibility:** Agile is a good fit for projects needing tailored approaches.
- **Small development teams:** Agile is ideal for small teams due to its simplicity and low resource requirements.

Selecting a SDLC Model for Project Integration:

- **Compatibility with distributed teams:** Agile is ideal for remote teams due to its support for online collaboration tools.
- **Prototyping support:** Spiral and Agile are suitable for projects requiring early prototypes and feedback.
- **Learning curve:** Waterfall is simpler for inexperienced teams, while Agile and Spiral require more learning time.
- **Availability of tools:** Tools like Jira, Trello (Agile), or Selenium enhance process efficiency.
- **Implementation cost:** Agile is cost-effective for small projects; Waterfall and Spiral are better for large projects but come with higher costs.
- **Market adoption:** Opt for widely adopted models to ensure access to resources and expertise.
- **Non-reliance on external resources:** Waterfall or Incremental Development is suitable for projects with limited external dependencies.

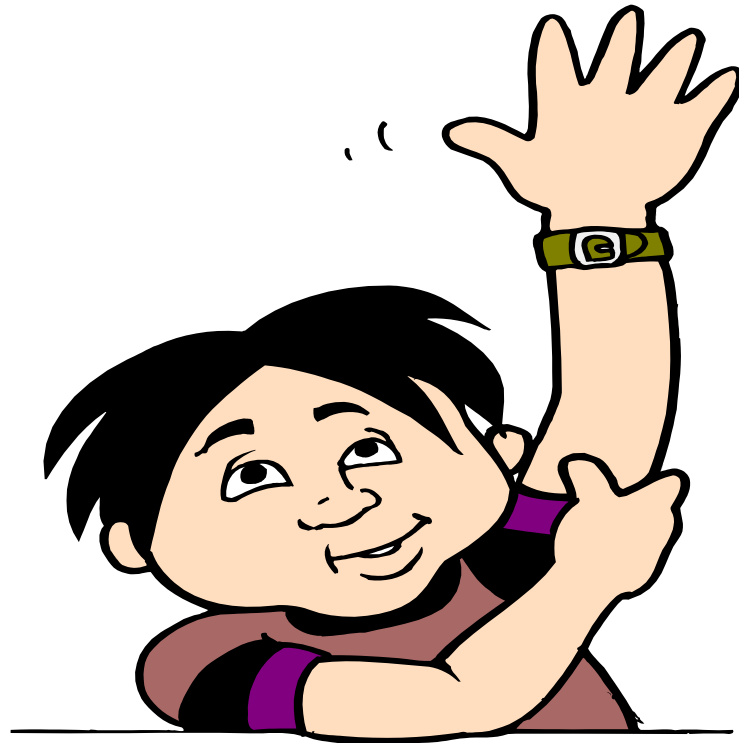
Heuristic 2

Build and maintain options as long as possible in the design and implementation of complex systems.
You will need them.

Summary

- Different types of system integration life cycles:
 - Waterfall
 - Spiral
 - Agile
- All life cycles have same stages, only differ in emphasis, duration, and timing
- System integration life cycle differs from Software Development Life Cycle:
 - SILC focuses on integrating legacy systems and managing dependencies between components.
 - Testing and maintenance are more complex in SILC than in SDLC.

QA



Questions and Discussion

1. Comparison of Waterfall, Spiral, and Scrum: Use a comparison table to highlight the strengths and weaknesses of each method.
2. Challenges in Scrum: Discuss the difficulties in integrating security and managing documentation within Scrum.

The end

THANK YOU